AI-Powered Crop Disease Detection and Advisory System

Assignment 6: Deployment Submission

1. Overview

The deployment phase represents the final step of the AI-Powered Crop Disease Detection and Advisory System, transitioning the refined YOLOv5 model from the research environment into a functional production setup. The goal was to make the model accessible to farmers and agricultural extension officers through a mobile advisory application. Deployment ensured that the trained model could deliver real-time, accurate predictions under real-world field conditions in rural Liberia.

The process involved serializing the optimized YOLOv5 model, integrating it into a TensorFlow Lite (TFLite) mobile environment, developing a RESTful API for backend predictions, and establishing robust monitoring, logging, and security protocols.

2. Model Serialization

After achieving the highest accuracy (96.4%) during the test phase, the YOLOv5 model was serialized for deployment to ensure efficient portability and fast inference.

- **Serialization Format:**
  The final model was exported to both `.pt` (PyTorch) and `.tflite` (TensorFlow Lite) formats to support flexibility between cloud-based and mobile applications.
- **Conversion Steps:**

```
# Export YOLOv5 model to TorchScript and TensorFlow Lite
import torch

model = torch.hub.load('ultralytics/yolov5', 'custom', path='best.pt', source='local')

# Convert to TorchScript
traced = torch.jit.trace(model, torch.randn(1, 3, 640, 640))
traced.save("best.torchscript.pt")

# Convert to ONNX for interoperability
model.export(format='onnx')

# Convert to TensorFlow Lite
model.export(format='tflite')
```

- **Efficiency Considerations:**
  The `.tflite` format was preferred for the Android application due to its compact

size and low latency. Quantization was applied (float16) to further reduce file size and optimize inference speed without noticeable performance loss.

3. Model Serving

The serialized model was deployed through two serving methods:

1. **Mobile Device Inference (On-Device):**
   The TensorFlow Lite model runs directly on Android smartphones using the React Native interface. This approach ensures offline access for farmers in areas with limited connectivity.
2. **Cloud-Based API (Optional):**
   The `.pt` model was also hosted on a Flask-based API running on Google Cloud Platform (GCP) for centralized data analysis and continuous improvement.

Example of the Flask serving endpoint:

```
from flask import Flask, request, jsonify
import torch
from PIL import Image
import io

app = Flask(__name__)

# Load serialized model
model = torch.hub.load('ultralytics/yolov5', 'custom', path='best.pt')

@app.route('/predict', methods=['POST'])
def predict():
    image_file = request.files['image']
    image = Image.open(io.BytesIO(image_file.read()))
    results = model(image)
    predictions = results.pandas().xyxy[0].to_dict(orient="records")
    return jsonify(predictions)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

4. API Integration

The API facilitates seamless communication between the mobile app frontend and the backend model service.

- **Endpoint:** `/predict`
- **Method:** `POST`
- **Input Format:**
  - Content-Type: `multipart/form-data`

    - Parameter: `image` (uploaded leaf photo)
- **Response Format (JSON):**
- [
- {
- "class": "Cassava Mosaic",
- "confidence": 0.96,
- "xmin": 100,
- "ymin": 120,
- "xmax": 400,
- "ymax": 460
- }
-]
- **Integration Example (React Native):**
- const formData = new FormData();
- formData.append('image', {
- uri: imageUri,
- type: 'image/jpeg',
- name: 'leaf.jpg'
- });
-
- fetch('https://crop-detect-api.onrender.com/predict', {
- method: 'POST',
- body: formData
- })
- .then(response => response.json())
- .then(data => console.log('Prediction:', data))
- .catch(error => console.error('Error:', error));

5. Security Considerations

Security was implemented to safeguard both data and system integrity during deployment.

- **Authentication:**
  API requests require an API key embedded within headers for authorized use.
- **Encryption:**
  HTTPS (SSL/TLS) was enforced for all API communications.
- **Data Privacy:**
  Uploaded images are processed in-memory and not stored permanently. Logs contain only anonymized metadata (timestamp, class type, confidence).
- **Mobile Data Security:**
  The TensorFlow Lite model is encrypted within the mobile app's assets to prevent reverse engineering or misuse.

Example API key validation:

```
from flask import abort

API_KEY = "secure-ml-key-2025"

@app.before_request
def authenticate():
    key = request.headers.get('x-api-key')
    if key != API_KEY:
        abort(401, description="Unauthorized")
```

6. Monitoring and Logging

Continuous monitoring ensures reliability, accuracy, and responsiveness of the deployed system.

- **Metrics Tracked:**
  - Model inference time (ms per image)
  - Prediction accuracy (based on field validation samples)
  - User activity and API call frequency
  - Error rates (e.g., failed inference or corrupted uploads)
- **Logging Implementation:**

```
import json, time

def log_event(event_type, data):
    log_entry = {
        "timestamp": time.strftime('%Y-%m-%d %H:%M:%S'),
        "event": event_type,
        "details": data
    }
    with open("deployment_logs.json", "a") as log_file:
        json.dump(log_entry, log_file)
        log_file.write("\n")
```

- **Monitoring Tools:**
  - **CloudWatch / Stackdriver:** Tracks uptime, latency, and usage patterns.
  - **Mobile Feedback Loop:** App users can report incorrect predictions for model retraining.
  - **Alert System:** Automated email alerts are triggered if API latency exceeds 500ms or if confidence scores drop below 85%.

Conclusion

The deployment phase successfully bridged the gap between research and field usability. The YOLOv5 model, optimized and converted to TensorFlow Lite, now powers a mobile advisory platform that operates both online and offline. The inclusion of robust security, monitoring, and cloud integration ensures sustainability and scalability of the system.

By delivering real-time, on-device crop disease detection, the project directly empowers farmers with actionable insights — reducing losses, improving yield, and advancing precision agriculture across Liberia and beyond.

Team Members

1. Nelson Papi Kolliesuah
2. Melvin Dauda
3. Michael Senkao
4. Lydia L. Labala
5. Cyrus A. B. Binda
6. McCess N. Belleh